ROBOTIC ARM — COMPLETE APP MASTER PLAN v3

Compact engineering specification + copy/paste development prompt series

Updated requirement: the controller may be **Arduino UNO R3, classic Nano, or Mega 2560**. The application, protocol and robot behavior remain board-independent.

LOCKED PRODUCT

Area	Decision
Robot	6-DOF 3D-printed arm; PCA9685; HC-05 Bluetooth Classic; six servos.
Controller	Arduino UNO R3 / classic Nano / Mega 2560 — all supported.
App	Flutter Android APK; modern robotics-controller UI.
Primary control	Manual game-like control: press/hold → move; release/neutral → stop.
Precision	Six target angles: Base, Shoulder, Elbow, Lower Arm, Wrist, Gripper.
Safety	STOP ALL / ESTOP, Arduino-side limits, invalid-command rejection, disconnect timeout.
Cloud	Firebase only for saved poses/sequences/settings/history; never real-time servo control.
V1 exclusions	AI, CV, IK, remote cloud servo control, complex analytics, large sequence editor.

VISUAL LOCK

Use the actual arm as the visual identity; short 3–5 s pick/place intro; clean light blue/cyan/white language; spacious cards; strong typography; professional icons. No purple gradients, excessive glassmorphism, emoji UI icons, fake metrics/reviews, AI-looking stock art, cursor animation or unnecessary effects.

MASTER RULE: Arduino is authoritative for physical safety and motion. Flutter is authoritative only for user interaction, targets and presentation.

HARDWARE + BOARD-AGNOSTIC CONTRACT

BOARD SUPPORT

Board	PCA9685 I²C	HC-05	Notes
UNO R3	A4 SDA / A5 SCL	SoftwareSerial on tested pins	Enough for V1; limited RAM.
Classic Nano	A4 SDA / A5 SCL	SoftwareSerial on tested pins	UNO-class firmware.
Mega 2560	20 SDA / 21 SCL	Prefer Serial1 (19 RX / 18 TX)	Best expansion/debug serial.

Use a **BoardProfile** so board-specific pins/configuration are isolated. Never redesign the Flutter side when changing UNO ↔ Nano ↔ Mega.

JOINT MAP

ID	Joint	PCA9685	Servo
0	BASE	CH0	MG995
1	SHOULDER	CH1	MG996R
2	ELBOW / UPPER ARM	CH2	MG996R
3	LOWER ARM	CH3	SG90
4	WRIST	CH4	SG90
5	GRIPPER	CH5	SG90

POWER GATE

External 5 V servo supply; never use Arduino USB/5 V for the servo rail. Common ground between Arduino, PCA9685 logic, HC-05 and servo supply. Validate rail voltage under load, wiring polarity, connectors and servo stall/peak current before simultaneous motion. Measure real mechanical limits; do not assume 0–180° is safe. Without position sensors, current angle = last commanded/calibrated angle, not measured physical angle.

PROMPT 01 — HARDWARE CONTRACT

```
You are the senior robotics firmware engineer. Create a board-agnostic hardware contract for a 6-DOF arm using PCA9685 + HC-05. Support Arduino UNO R3, classic Nano and Mega 2560 through BoardProfile. Keep joint IDs 0–5 and PCA9685 mapping fixed. Separate pins, calibration, limits and protocol from application logic. Do not invent mechanical limits. Output wiring assumptions, BoardProfile, compile-time board selection, diagnostics and a safe servo-test plan.
```

ARDUINO FIRMWARE FOUNDATION

ARCHITECTURE

BoardProfile → Serial/Bluetooth → ProtocolParser → SafetyController → MotionController → PCA9685Driver → Servos. RobotState is the single runtime state source.

State	Purpose
connection / handshake	Only accept motion after valid handshake.
estop / motionState	STOPPED, MANUAL, MOVING_TO_TARGET, HOMING, ESTOP.
currentCommanded[6]	Last commanded/calibrated angle per joint.
target[6]	Validated target pose.
limits[6]	Min/max enforced on Arduino.
speedProfile	Slow / Normal / Fast.
lastCommandTime	Bluetooth/command timeout.
lastError	Protocol or safety rejection reason.

PROMPT 02 — FIRMWARE CORE

```
Implement Arduino firmware for UNO/Nano/Mega with PCA9685, RobotState, parser, limits, ESTOP, STOP, timeout, safe startup and non-blocking motion. Use fixed-size structures suitable for UNO/Nano RAM. Modular files: board_profile, protocol, safety, motion, servo_driver, config, main. Avoid dynamic allocation for routine commands. Return explicit ACK/ERR. Do not claim measured angles without sensors.
```

PROMPT 03 — SERVO CALIBRATION

```
Add safe per-joint calibration: mechanical reference, center/offset, minAngle, maxAngle, safeHome and default speed. Test one joint at a time at low speed. Reject out-of-range commands before PWM. Add a serial command that prints the active calibration table. Never assume 90° is physical home.
```

PROMPT 04 — SERIAL DIAGNOSTICS

```
Create a diagnostic mode for UNO/Nano/Mega that reports board profile, PCA9685 availability, HC-05 serial state, configured joint limits and last command. Keep diagnostics separate from the production command protocol and avoid blocking the control loop.
```

BLUETOOTH PROTOCOL + SAFETY

PROTOCOL V1 — LINE DELIMITED

Command	Meaning
HELLO	Start handshake.
STATUS?	Request robot state.
READY_ACK	Confirm application received ready/state.
SEL:0..5	Select joint.
SPD:1|2|3	Slow / Normal / Fast.
J+:id / J-:id	Manual directional command.
POSE:b,s,e,l,w,g	Set validated target pose.
MOVE	Start coordinated target move.
STOP	Immediate normal stop.
ESTOP	Emergency stop.
HOME	Move to validated safe-home pose.

HANDSHAKE

APP → HELLO → Arduino → ROBOT_6DOF_READY v1 → APP → STATUS? → Arduino → STATE... → APP → READY_ACK. Only after this is the robot operational.

SAFETY

Arduino rejects motion before handshake; stops after communication timeout; validates every target against limits; rejects malformed/unknown/stale/out-of-state commands; STOP is always actionable; ESTOP requires intentional recovery. Manual release/joystick-neutral means no continued movement.

PROMPT 05 — PROTOCOL + SAFETY

```
Implement the versioned line protocol above with deterministic ACK/ERR, handshake state machine, timeout, STOP and ESTOP. Arduino is authoritative. A disconnected phone must not leave a servo moving indefinitely. Provide serial tests for every command, malformed input, limit violation, motion-before-ready, timeout and ESTOP recovery. Keep protocol identical across UNO/Nano/Mega.
```

PROMPT 06 — PROTOCOL TESTER

```
Create a simple serial/Bluetooth test harness that can send each protocol command, display raw responses, timestamp replies, flag ACK/ERR and run repeatable handshake/timeout/limit tests. It must never automatically issue unsafe motion commands without explicit operator confirmation.
```

MOTION ENGINE + HARDWARE TEST GATES

MOTION MODEL

Manual motion is incremental velocity-style control: joystick displacement → speed; dead-zone prevents chatter; neutral/release stops. Pose movement uses coordinated interpolation so all six joints reach target smoothly instead of six abrupt jumps.

Gate	Test	Pass condition
G1	One servo low speed	Correct direction; no overtravel/buzzing.
G2	Joint min/max	Stops/rejects at configured limits.
G3	STOP during motion	Motion stops immediately.
G4	Bluetooth disconnect	Motion stops after timeout.
G5	POSE move	All axes interpolate smoothly.
G6	Power/load	5 V rail remains acceptable.
G7	ESTOP	Motion disabled; intentional recovery.
G8	Repeated commands	No runaway/parser lockup/memory growth.

PROMPT 07 — MOTION ENGINE

```
Implement a non-blocking six-joint motion engine. Given current commanded angles and validated target, calculate a common duration and update each joint incrementally at a stable interval. Preserve limits and speed. STOP cancels motion immediately. Avoid delay()-based blocking loops. Add manual velocity-style movement with dead-zone and speed profiles.
```

PROMPT 08 — HARDWARE TEST HARNESS

```
Create a safe firmware test mode: one PCA9685 channel → one joint → grouped motion. Start conservative. Print joint ID, requested angle, clamped angle and state. Include operator checks for physical support, clearance, external supply, common ground and STOP access. Do not enable full six-axis movement until all gates pass.
```

PROMPT 09 — POWER/LOAD CHECK

```
Add a hardware validation checklist for the actual 5 V servo supply, connectors, common ground and servo current demand. Define what must be measured under idle, one-servo, multi-servo and near-load conditions. Do not assume the nominal 5 V/10 A supply is sufficient without measurement.
```

FLUTTER FOUNDATION + APP STRUCTURE

ARCHITECTURE

```
lib/ core/ constants, theme, safety, utilities models/ joint, robot_state, pose, sequence bluetooth/ manager, connection, protocol controllers/ robot, motion, safety services/ storage, firebase, preset screens/ splash, home, control, pose, saved, settings widgets/ joystick, angle_row, robot_view, status, estop main.dart
```

Use Riverpod or Provider for reactive state. Bluetooth and RobotState live outside widgets. Widgets render state and emit high-level intents; they do not contain servo protocol logic.

Screen	Purpose
Launch	3–5 s arm pick/place intro; reduced-motion/static fallback.
Home	Arm visual, connection/ready state, six commanded angles, Home, Control, STOP.
Control	Primary manual game-like operation.
Pose	Current vs Target six-angle editor; Preview → Move.
Saved	Saved poses; sequences after core stability.
Settings	Bluetooth, calibration, limits, speed, robot info, legal links.

PROMPT 10 — FLUTTER FOUNDATION

```
Create the Flutter Android app architecture for this robotic-arm controller. Use typed models, centralized RobotState, navigation for Launch/Home/Control/Pose/Saved/Settings and reusable widgets. Keep Firebase isolated. Do not implement servo behavior in widgets. Add disabled/loading/error states for Bluetooth and readiness. Use the locked clean robotics visual language; no purple gradients or fake metrics.
```

PROMPT 11 — DESIGN SYSTEM

```
Create the design system: typography scale, spacing, cards, professional iconography, status colors, button hierarchy, joystick styling and persistent STOP treatment. Keep light blue/cyan/white product direction, restrained rounded cards and strong readability. Design for real phone touch targets and safe one-handed STOP access.
```

BLUETOOTH + MANUAL GAME CONTROLLER

BLUETOOTH FLOW

Scan/pair → connect HC-05 SPP → HELLO → ROBOT_6DOF_READY → STATUS? → STATE → READY_ACK → Ready. Reconnect repeats the handshake. Socket-connected ≠ robot-ready.

Input	Joint
Left joystick X	Base
Left joystick Y	Shoulder
Right joystick X	Elbow
Right joystick Y	Lower Arm
Wrist dedicated rocker/control	Wrist
Gripper dedicated open/close	Gripper
Fine Joint mode	Select any joint for precision manual control

Manual behavior: displacement/press-and-hold starts motion; release/neutral stops. Dead-zone prevents chatter. Small displacement = precision; larger displacement = faster. Rate-limit outbound commands. STOP ALL is permanently reachable.

PROMPT 12 — BLUETOOTH LAYER

```
Implement Android Bluetooth Classic SPP for HC-05 using a maintained Flutter package compatible with the chosen Flutter version. Encapsulate discovery, pairing, connect/disconnect, framing, receive stream, timeout and reconnect. Implement the exact handshake and expose ConnectionState separately from RobotReady. Never mark Ready merely because a socket connected. Handle Android permissions and clear errors.
```

PROMPT 13 — MANUAL CONTROLLER

```
Build the Control screen as a game-like robotic-arm controller with the locked mapping: left X Base, left Y Shoulder, right X Elbow, right Y Lower Arm; dedicated Wrist and Gripper; Fine Joint mode. Implement dead-zone, release/neutral stop, Slow/Normal/Fast, command rate limiting and persistent STOP ALL. Disable motion controls unless RobotReady is true.
```

PROMPT 14 — CONNECTION FAILURE UX

```
Design connection-loss UX that immediately shows Disconnected/Not Ready, disables motion controls, displays the recovery path and never implies the arm is safe merely because the UI is open. Reconnect must repeat the full handshake. Reflect Arduino timeout/STOP state when the next STATUS response arrives.
```

POSE / ANGLE MODE + SAVED POSITIONS

POSE SCREEN

Show CURRENT and TARGET separately for all six joints. Current means last commanded/calibrated state unless real feedback sensors are later added. Validate locally for immediate feedback, then validate again on Arduino.

Joint	Current	Target
Base	commanded	editable
Shoulder	commanded	editable
Elbow	commanded	editable
Lower Arm	commanded	editable
Wrist	commanded	editable
Gripper	commanded	editable

Workflow: Edit → validate → Preview → explicit MOVE TO TARGET → Arduino coordinated motion → state/progress → Target Reached/Error. Slider edits alone never move the arm.

PROMPT 15 — POSE MODE

```
Build six-angle Pose mode with per-joint min/max, Current vs Target, Preview and explicit Move to Target. Preview must never command hardware.
Move sends POSE then MOVE and shows motion state, completion, errors and STOP. Do not claim physical angle measurement.
```

PROMPT 16 — SAVED POSES

```
Implement saved poses with name, six angles, timestamp and optional description. Support Save Current, Load/Edit, Preview and Move. Store locally
first; isolate Firebase behind a service so the core controller works offline. Never let cloud reads/writes trigger real-time servo movement.
```

PROMPT 17 — CALIBRATION + LIMIT UI

```
Create Settings screens for calibration and joint limits. Distinguish mechanical reference, command offset, safe minimum, safe maximum, safe home
and default speed. Require deliberate confirmation before safety-limit changes. Arduino remains final authority.
```

FIREBASE, DASHBOARD, POLISH + QA

PROMPT 18 — FIREBASE

```
Add isolated Firebase storage for saved poses, optional sequences and optional history. Make the system offline-first: Bluetooth control continues without internet. Define minimal Firestore schema and security rules. Do not put Firebase in the command path.
```

PROMPT 19 — HOME DASHBOARD

```
Build Home around a genuine robotics product: actual arm visual, HC-05 connection, Robot Ready state, six commanded angles, Home, Control shortcut and persistent STOP ALL. Never fabricate telemetry. Home uses the validated HOME command and shows motion state until completion.
```

PROMPT 20 — LAUNCH ANIMATION

```
Create a short 3–5 second introduction using the actual arm asset: HOME → approach small block → gripper closes → lift → move → gripper opens → place → dashboard. Keep it restrained and purposeful. Add reduced-motion/static fallback. Do not make the launch animation delay safety controls or hide connection state.
```

PROMPT 21 — FINAL UI POLISH

```
Polish without changing architecture. Use light blue/cyan/white, spacious layout, clear typography, professional icons, restrained rounded cards and subtle transitions. Remove purple gradients, excessive glass, emoji controls, fake metrics/reviews, vague marketing copy, cursor animations and unnecessary effects. Verify touch targets and STOP visibility on the actual phone.
```

PROMPT 22 — FULL QA

```
Create and execute a test matrix for install, Bluetooth permissions, pairing, handshake, reconnect, Arduino reboot, malformed commands, limits, STOP, ESTOP, disconnect timeout, release/neutral stop, pose preview, coordinated motion, calibration persistence, saved poses, Firebase offline behavior, app backgrounding, screen rotation, low battery and release build. Record expected results and pass/fail evidence.
```

PROMPT 23 — RELEASE

```
Prepare the production Android APK: remove debug UI/logging, verify permissions, verify legal links, test release Bluetooth behavior, sign the APK, document supported Arduino boards and wiring assumptions, and create a one-page operator/demo checklist.
```

MASTER BUILD PROMPT SERIES — EXECUTE IN ORDER

Do not paste one giant prompt and expect a reliable system. Use the phases below sequentially in the same coding workspace. After each phase: compile → run tests → inspect actual behavior → fix → commit/tag → continue.

PHASE A — HARDWARE + FIRMWARE

01 Hardware Contract → 02 Firmware Core → 03 Servo Calibration → 04 Diagnostics → 05 Protocol/Safety → 06 Protocol Tester → 07 Motion Engine → 08 Hardware Test Harness → 09 Power/Load Check.

PHASE B — FLUTTER CORE

10 Flutter Foundation → 11 Design System → 12 Bluetooth Layer → 13 Manual Controller → 14 Connection Failure UX.

PHASE C — PRECISION CONTROL

15 Pose Mode → 16 Saved Poses → 17 Calibration/Limit UI.

PHASE D — PRODUCT COMPLETION

18 Firebase → 19 Home Dashboard → 20 Launch Animation → 21 UI Polish → 22 Full QA → 23 Release.

MASTER PROMPT — USE BEFORE EVERY PHASE

```
You are the senior robotics + Flutter engineer continuing an existing 6-DOF robotic-arm project. Preserve all locked requirements from this master plan. The physical stack is HC-05 → Arduino UNO/Nano/Mega → PCA9685 → six servos. Arduino is authoritative for safety, limits, STOP/ESTOP, timeout and motion. Flutter owns UI, Bluetooth, targets and storage. Never invent mechanical limits or measured angles. Do not redesign locked behavior without identifying the engineering reason. Implement only the current phase, keep interfaces compatible with the previous phase, test before claiming completion, and report exact files changed, tests run, failures found and next step.
```

FINAL ACCEPTANCE CHECKLIST

Area	Must pass
Boards	Logical firmware works on UNO, classic Nano and Mega through BoardProfile.
Control	Press/hold or joystick displacement moves; release/neutral stops.
Safety	STOP/ESTOP, limits, invalid-command rejection and disconnect timeout work.
Bluetooth	Full handshake required; reconnect repeats handshake.
Pose	Current vs Target; Preview never moves; Move uses coordinated interpolation.
Data	Saved poses work offline; Firebase is optional and never in servo path.
UI	Professional robotics product; actual arm visual; no prohibited visual patterns.
Release	Clean signed APK, legal links, operator/demo checklist.

FINAL PRINCIPLE

Reliable physical control first. Precision second. Product polish third. Cloud and extra features only after the machine is safe and stable.

SOURCE BASIS

Grounded in the supplied Robotic Arm Control project specification: Flutter + HC-05 architecture, manual press/hold control, six-angle pose control, handshake/safety, saved poses, restrained visual direction and Firebase separation. UNO/Nano/Mega flexibility is explicitly incorporated in this revision.